

Backend Architecture and Developer Guide

The backend code is organized around a three-tier architecture, which facilitates testing and managing complex database operations.

Getting started

1. Environment Setup:

- Ensure you have Python 3.13+ installed.
- Install the [uv](#) package manager if you haven't installed yet.
- Ensure you have configured PostgreSQL, Redis, and Keycloak correctly.

2. Install dependencies:

- Run `uv sync` to install the required dependencies from [pyproject.toml](#).

3. Setup environment variables:

- Copy the `.env_template` file to `.env` in the root of the backend directory and adjust the values as needed:

```
cp .env_template .env
```

```
POSTGRES_USER=  
POSTGRES_PASSWORD=  
POSTGRES_HOST=  
POSTGRES_PORT=  
POSTGRES_DB=  
  
KEYCLOAK_URL=  
KEYCLOAK_REALM_NAME=  
KEYCLOAK_CLIENT_ID=  
KEYCLOAK_ALGORITHM=  
  
REDIS_HOST=  
REDIS_PORT=  
REDIS_DB=  
REDIS_PASSWORD=  
REDIS_STREAM=  
REDIS_GROUP=  
REDIS_CONSUMER=
```

4. Run backend:

- Start the FastAPI development server with auto-reload:

```
uv run uvicorn src.api.main:app --reload
```

(Note: Adjust `src.main:app` if your FastAPI application instance is located elsewhere)

Layered Architecture

1. **Models (`src/models/`):** Data layer mapped to a relational database (PostgreSQL) using SQLAlchemy 2.0. It contains class definitions (entities), table structures, and relationships between them (e.g., `UserProfile`, `Technology`).
 2. **Schemas (`src/schemas/`):** Data validation models based on Pydantic. They are responsible for defining the shape of input (Request) and output (Response) data, acting as DTOs (Data Transfer Objects). They isolate the application from invalid external data (e.g., filtering out redundant keys, verifying types) and ensure type safety before the data goes deeper into the application.
 3. **Routers (`src/api/`):** API Controllers (FastAPI). They act as the main entry point for HTTP requests. They are solely responsible for routing, permission verification using *Dependency Injection* (e.g., JWT token validation), and passing valid schemas (DTOs) to the Service. They are intentionally stripped of business logic.
 4. **Services (`src/services/`):** **Business Logic** layer. Services receive data from routers and perform the main "work" of the application. They make decisions, apply advanced rules, and orchestrate logic using multiple repositories. They operate on schemas and models but do not create SQL queries.
 5. **Repositories (`src/repositories/`):** **Data Access Layer (DAL)**. The only layer that directly interacts with database queries (CRUD operations, abstraction over `select()`, `insert()`, `update()`). Repositories operate on models (`src/models/`) and return them to services, which simplifies session management.
-

Services Overview

The services layer (`src/services/`) is the heart of the application, where all the system's "intelligence" is implemented. Controllers (routers) only pass validated data here, and services orchestrate the cooperation between the database (repositories) and the rest of the logic.

Main services and their brief description:

1. **JobOffersService:** Manages the full lifecycle of job offers. Responsible for verifying uniqueness and saving normalized data from scrapers (in collaboration with the lookups layer), as well as handling advanced filters coming from the API (including dynamic modification of filters based on user login status).
2. **UserProfileService:** Handles the management of profiles associated with Keycloak accounts. Implements secure UPSERT (create or update) logic, deciding whether to create or update a profile based on the database state, and properly assigning/overwriting many-to-many relationships for user technologies.
3. **TechExtractorService:** A utility service providing algorithms for processing text from CV files (.pdf, .docx). Responsible for intelligent keyword extraction, punctuation removal, deduplication, and matching (fuzzy matching) of possessed skills to standardized technologies in the database.
4. **LookupsService:** A universal service providing access to dictionary tables (e.g., `Technology`, `Location`, `Site`). It acts as a central relationship resolution mechanism - safely translating identifiers (UUIDs) sent in requests to the corresponding database objects and allowing for the "on-the-fly" creation of missing entities to maintain foreign key integrity.

5. **AuthService**: The main integration point with the external identity provider (Keycloak). Decodes and cryptographically verifies JWT tokens sent by clients. Extracts identity data from them (e.g., Keycloak UUID, email address) and coordinates with the user repository to return the appropriate **User** model. Throws appropriate exceptions (e.g., **401 Unauthorized**) when the token is invalid or expired.
6. **CVService**: Coordinates the processing of incoming CV files from clients. Verifies the correctness of document formats (supporting only **.pdf** and **.docx**), and then delegates text extraction and technology mapping to the **TechExtractorService**. Returns a ready and validated list of matching technology entities to the router.

All these services are utilized via Dependency Injection in API endpoints (FastAPI) and maintain complete isolation from external HTTP requests.

Endpoints Overview (Routers)

The controllers layer (**src/api/v1/**) is the direct application interface for HTTP clients. Each file in this directory (a so-called router) groups thematically related REST API endpoints, whose task is to receive a request, invoke dependency injection (e.g., authorization verification), and pass validated schemas to the appropriate services.

The main logical groupings of endpoints are:

1. **job_offers.py (/job-offers)**: Provides an interface for browsing and searching job postings. The key endpoint is **GET /job-offers/get_offer_filter**, which modifies the behavior of **JobOffersService** based on provided query parameters (e.g., pagination, technologies) and token verification.
2. **user_profiles.py (/me)**: Endpoints used to manage the data of the logged-in user. Secured by a required JWT token (**Depends(get_current_user)**). Used to read the private profile and perform modification/creation (UPSERT) operations on it.
3. **cv.py (/cv)**: Endpoints adapted to accept streamed file uploads (**UploadFile**). They secure the application by verifying extensions (e.g., rejecting everything except **.pdf** and **.docx**), to finally return a JSON list of recognized technologies.
4. **lookups.py (/lookups)**: A set of endpoints facilitating the building of the client interface. They return application dictionaries as lists (all supported technologies, companies, experience levels), which are necessary to display frontend dropdown lists and filtering options.

*A complete and up-to-date list of all endpoints is available in the interactive Swagger UI at **/docs**.*

Key Business Mechanisms

Saving Job Offers from Scrapers (**JobOffersService**)

Although a separate background module (Worker integrated with scrapers) is responsible for listening and processing external data, **our application (Service Layer)** handles their final validation and storage.

1. Normalized data flows into our defined input schemas (e.g., **JobOfferScraperCreate**), which act as a validation barrier.
2. Then **JobOffersService** takes responsibility: it verifies the uniqueness of offers (e.g., by **url**), uses the Lookups layer (to assign or create missing dictionary entries), and instructs the Repositories to execute

the save operation using many-to-many relationships (e.g., attaching technologies to the offer).

Filtering and Exposing Job Offers (API → Frontend)

Our API exposes endpoints that allow the frontend to perform advanced searching and filtering of job offers. This mechanism also takes into account access differentiation based on the authorization state:

1. **Parameter Validation (Schemas):** Frontend requests (Query Parameters) are automatically mapped and validated using a Pydantic schema (`JobOfferFilter`), which guarantees type correctness (e.g., lists of technology IDs, salary ranges, pagination).
2. **Access Differentiation (Routers):** We use `Depends(get_optional_current_user)` in the router. If the user is not logged in, we automatically modify their filters, limiting the visibility of offers to only one, default source (defined in `UnregisteredUserSettings`). Logged-in users retain access to the full pool.
3. **Dynamic Queries:** Validated and adjusted filters pass through `JobOffersService` to the Repository layer. There, they are used to dynamically build advanced queries in SQLAlchemy 2.0, and the results return to the client in the safe form of an output model (`JobOfferResponse`).

Technology Extraction from CV (`TechExtractorService`)

The application provides logic for automatically "reading" skills from a user's CV submitted as a .pdf or .docx file, and matching them against the technology dictionary in the database.

How does it work?

1. **Text Cleaning:** The `_extract_words_from_text` method removes punctuation marks, single characters, and bare numbers using regular expressions (Regex).
2. **Fuzzy Matching:** The service fetches a cache of all available technologies, and then compares the words extracted from the CV with the database, allowing for minor typos (e.g., matching the word "Pyton" to "Python").
3. **Deduplication:** Extracted technologies are uniquely filtered so that a single technology mentioned multiple times in the CV is assigned only once.

Authorization and Security (Keycloak)

The entire authentication and identity management mechanism is delegated to the external **Keycloak** server. Our database does not store passwords, relying solely on `user_id` identifiers (UUIDs from Keycloak).

Permission verification happens on the first line of defense, at the **Routers** layer, utilizing FastAPI's built-in Dependency Injection system:

1. **JWT Token Validation:** Every request to a secured endpoint must have a JWT token in the `Authorization` header. Our API verifies its validity asynchronously using a public key.
2. **Dependency Injection:**
 - `Depends(get_current_user)` – Strictly enforces authorization. If the token is missing or invalid, FastAPI immediately rejects the request (401 code). This protects, among others, endpoints operating on private user profiles.
 - `Depends(get_optional_current_user)` – Optional dependency. Returns the logged-in user object or `None`. Used where a given endpoint (e.g., reading offers) has different logic depending on the login state. Thanks to this, the token validation logic is extracted before the business code is called, and the Services layer can safely assume that the received user object is valid.

User Profile Management (`UserProfileService`)

Every authenticated user has the ability to create and edit their own profile (containing, among others, experience level and a list of technologies). From the application's perspective, this process is implemented as an **UPSERT** (Update or Insert) operation, which requires coordinating several layers:

1. **Data Validation (Schemas):** The client sends data (e.g., the `UserProfileUpdate` model), which contains a list of identifiers (UUIDs) of selected technologies and the experience level. Pydantic ensures the correct format of these identifiers.
2. **Relationship Resolution (Lookups):** Before the profile is saved, `UserProfileService` must translate the provided "raw" identifiers into full relational objects from the database. For this purpose, it communicates with the `LookupsService` layer, ensuring that the provided technologies actually exist in the dictionaries.
3. **UPSERT Logic in Service:** The service verifies whether a profile already exists in the database for the given `user_id` (tokenized in Keycloak):
 - If the profile does not exist – it maps the schema to a new `UserProfile` entity and delegates the `create` operation to the repository.
 - If the profile exists – it modifies it accordingly and calls the `update` operation, safely overwriting many-to-many relationship bindings in the repository (e.g., updating technologies attached to the user).

Testing Convention

Our tests (`pytest` + `pytest-asyncio`) are fully isolated from the real database. They rely on **Mocking** individual layers.

If you are testing, for example, `UserProfileService`, the entire Repository layer (`mock_repo = AsyncMock()`) and `LookupsService` (`mock_lookups`) are simulated. We only check the business logic - e.g., whether the service correctly uses validated data from the `Schema` object, maps it to an entity, and orders the execution of "create" or "update" in the repository (depending on whether the profile model already existed in the database).

What is Covered by Tests?

Our unit tests focus strictly on business rules, validation, and edge cases. Here is an overview of the tested scenarios across our main services:

1. `TechExtractorService` Tests:

- **Regex Tokenization:** Verifies that punctuation and numbers are correctly stripped, and standalone characters are ignored.
- **Fuzzy Matching:** Ensures that typos (e.g., matching "Pyton" to "Python") and case variations are correctly mapped to database technologies.
- **Deduplication:** Confirms that a skill mentioned multiple times in a single CV results in only one technology entity being attached.

2. `JobOffersService` Tests:

- **Validation Rules:** Checks if the service correctly blocks invalid data before hitting the database (e.g., throwing `ValidationError` when `salary_min > salary_max` or when titles are empty).
- **Conflict Handling:** Verifies that attempting to create a job offer with an already existing `url` correctly throws a `RecordAlreadyExistsError`.
- **Scraper Ingestion Flow:** Simulates the complex process of saving an offer from a scraper, ensuring that the service resolves string names to lookups (via `LookupsService` mappings) and ultimately triggers a single atomic transaction (`create_with_relationships`) in the repository.

3. `UserProfileService` Tests:

- **UPSERT Logic:** Mocks the repository to simulate two core decision-making scenarios:
 - When a profile *exists*, it verifies that the `update_profile` method is triggered.
 - When a profile *does not exist*, it verifies that the `create_profile` method is triggered.
- **Relationship Handling:** Ensures that provided UUIDs from the API are properly resolved to full `Technology` objects using `LookupsService` before being passed to the repository layer.
- **Not Found Scenarios:** Validates that querying a non-existent profile correctly raises a `RecordNotFoundError`.